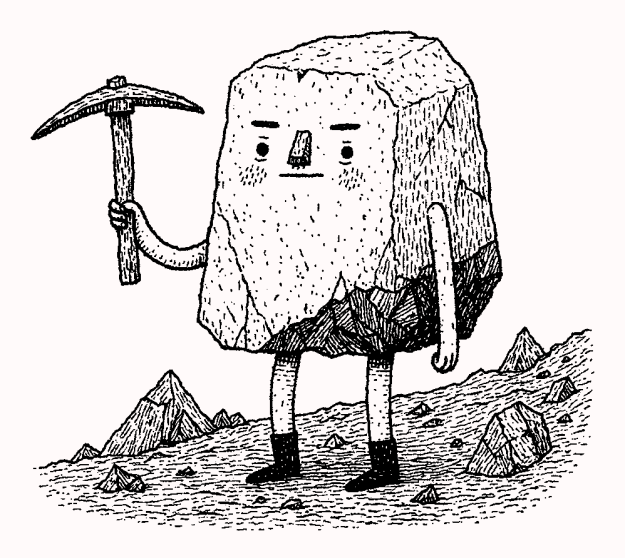
Autolith

A live, self-modifiable, introspectable terminal agent runtime inside a Common Lisp image.

Website	lambda-symbolics.com/autolith
Source	github.com/lambda-symbolics/autolith
Author	Lukáš Hozda
Status	in development since June 2026



The status quo (?)

Contemporary agent harnesses

- Fixed orchestration loop
- Opaque state and tool execution
- No budgets and capability limits
- Often just one provider, or a predefined list

Autolithic alternative

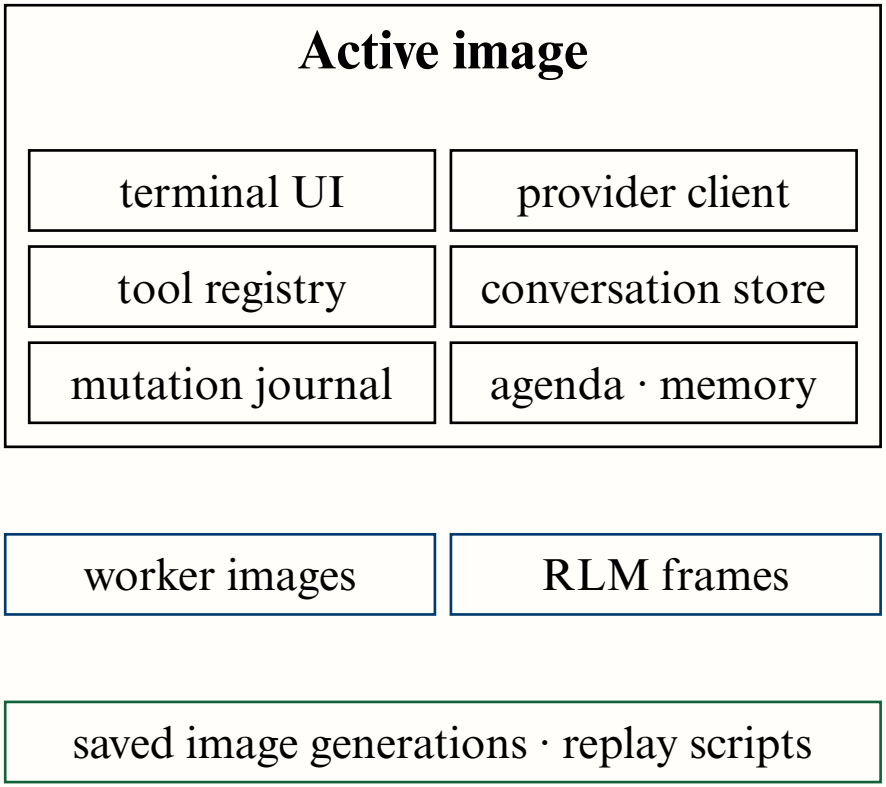
- A live, programmable runtime transparently available to the model
- State, tools, policy, ledgers: inspectable data
- Budgets, capabilities, isolation: explicit objects
- Providers are interchangeable clients

What does an agent look like built as a live computing system, rather than a chat wrapper?

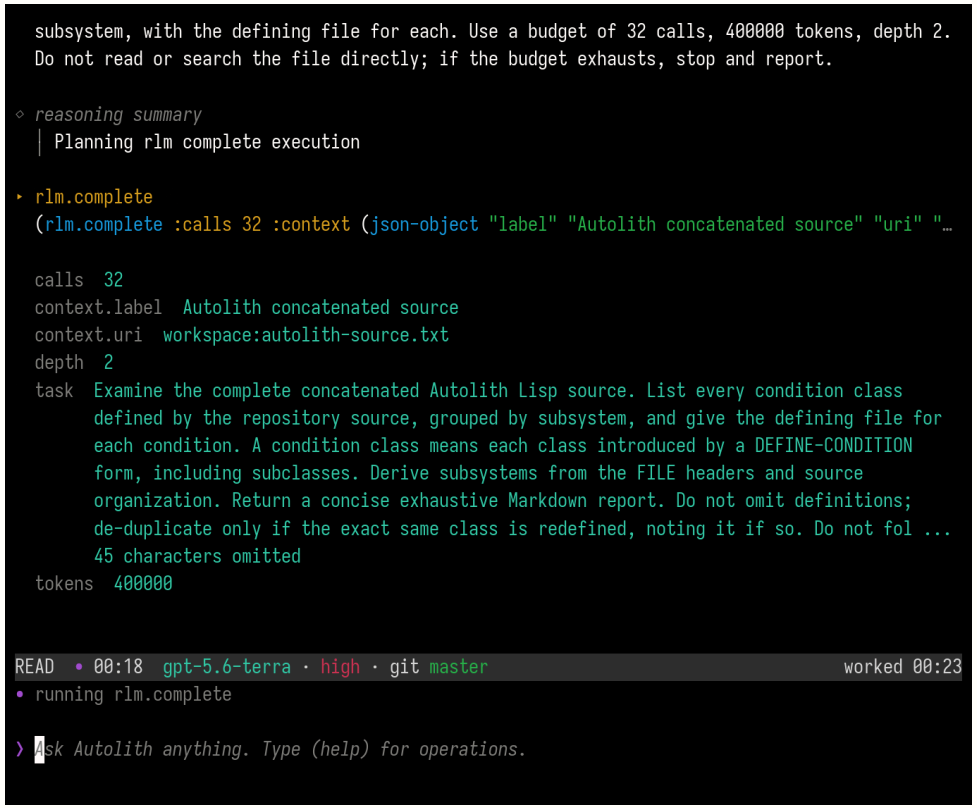
Models are now capable enough that the runtime around them increasingly determines their autonomy, reliability, and controllability.

Contemporary Autolith (what we have)

- Live self-modification tools: redefine, exercise, commit, append-only mutation journal
- Crash capsules, pristine recovery image, image generations
- 7 providers, plus any OpenAI-compatible endpoint
- Isolated workers, confined resources
- Recursive Language Model (RLM) tools
- 50+ people on Zulip
- Linux, macOS, FreeBSD, NetBSD, OpenBSD



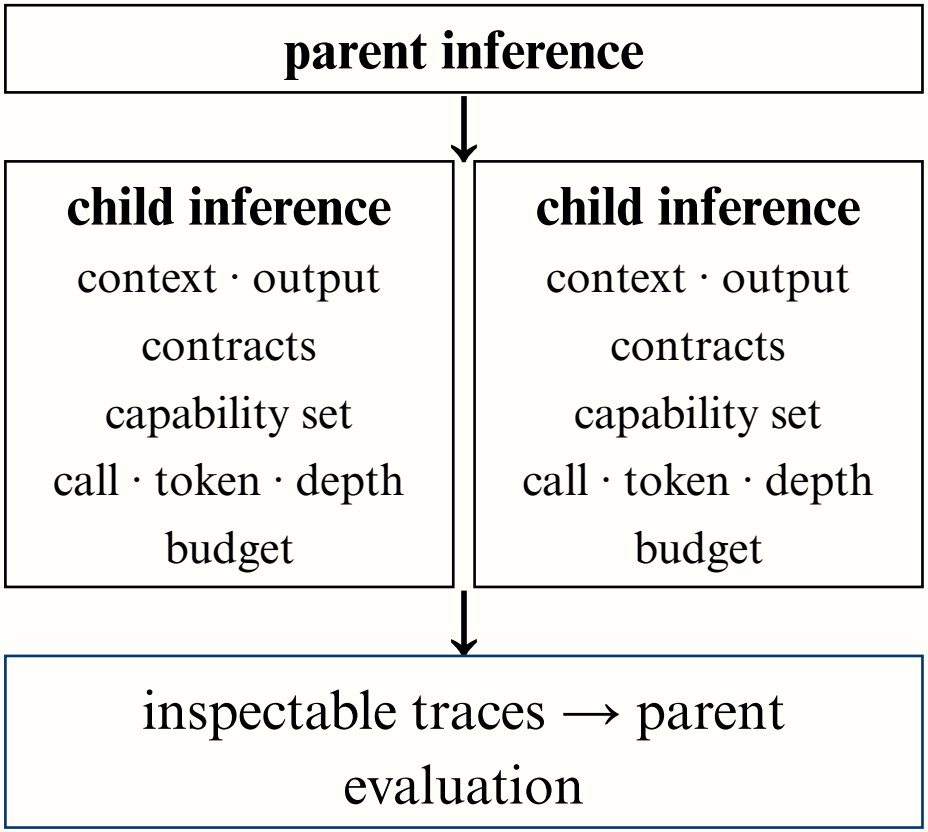
Example RLM call



- The task is to ingest the entire Autolith codebase and discover and describe condition classes
- An `rlm.complete` call is issued: 32 child calls, 400k tokens, at depth 2
- 3.1 MB example corpus enters as `workspace:autolith-source.txt` as programmable data to RLM instances
- Completed in 3 min 15 s; full recording is on the project page

Recorded (v0.35), unedited.

Recursive inference (RLM)



- Call, token, and depth budgets are shared down the tree
- Parallel children are exceuted against one pool; output metered in tranches
- RLM Children inherit nothing, the corpus lives in an isolated environment
- The model sees label, size, digest; works via slices, searches, sub-inferences
- Every frame leaves a replayable trace (`inference:ID`)
- Budget exhaustion triggers a typed condition the parent agent can work with

Autolith openness

- Autolith is licensed under the very permissive ISC license
- Runtime and accumulated state inspectable down to source
- Provider-independent: subscriptions, API keys, self-hosted endpoints
- Pieces ship as libraries: image generations, budgets, provider clients
  - We want to share as much as possible with the broader ecosystem to encourage development
- Execution and capability policy is user-editable code
- A reference implementation for agent researchers, with or without Autolith itself
- Closing it would remove an inspectable implementation that researchers can modify down to its inference and execution policy
-

What the grant unlocks

Six months of focus time to turn the experimental RLM work into a robust, reusable subsystem:

- enforceable shared limits: tokens, calls, depth, context, capabilities
- better sandboxing, recovery, inspectable execution traces
- local and provider-independent models
- reproducible benchmarks against other agent harnesses
- standalone libraries from generally useful components
- documentation, packaging, release infrastructure

R&D time; primarily mine, partly Jack Chakany's	$38k
Model inference, benchmark runs, compute	$6k
Release infrastructure, hosting, hardware	$3k
External security review, research travel	$3k

The broader question

Can AI agents look more like programmable computers: systems which can be modified and introspected, whose reasoning, state, resource use and actions can be observed and deliberately constrained?